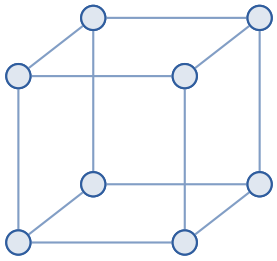
Algorithmique & Architectures parallèles

Résumé de cours complet



1. Introduction au calcul parallèle
2. Architectures parallèles
3. Multi-cœurs, GPU et CUDA
4. Réseaux d'interconnexion
5. Communications (o2o, o2a...)
6. Grain, degré, T_{opt} et P_{opt}
7. Évaluation des performances
8. Formes et sources de parallélisme
9. Méthodologie : systèmes de tâches
10. Programmation avec OpenMP
11. Fiche formules & questions de révision

Table des matières

1 Introduction au calcul parallèle **3**

1.1 Pourquoi le parallélisme est devenu indispensable 3

1.2 Temps séquentiel vs temps parallèle 4

2 Architectures parallèles **5**

2.1 Classification de Flynn (1966) 5

2.2 SIMD et MIMD 5

2.3 Mémoire partagée vs mémoire distribuée 6

2.4 Classement de Leopold (2001) 7

2.5 Système parallèle vs système distribué 7

2.6 Clusters et grilles 7

3 Multi-cœurs, GPU et CUDA **9**

3.1 Processeurs multi-cœurs 9

3.2 GPU 9

3.3 CUDA (NVIDIA) 9

4 Réseaux d'interconnexion **11**

4.1 Machines à mémoire partagée (réseaux dynamiques) 11

4.2 Machines à mémoire distribuée (réseaux statiques) 12

4.3 Tableau récapitulatif des topologies 16

5 Communications dans un réseau à mémoire distribuée **18**

5.1 Temps de communication 18

5.2 Types de communication 18

5.3 Types de liens et dispositif de communication 18

5.4 Réseau linéaire (p processeurs) 19

5.5 Anneau (p processeurs) 19

5.6 Grille 2D ($q_1 \times q_2$) 21

6 Grain, degré de parallélisme, T_{opt} et P_{opt} **22**

6.1 Temps et nombre de processeurs optimaux 22

7 Évaluation des performances **24**

7.1 Loi d'Amdahl 24

8 Formes et sources de parallélisme 25

8.1 Formes de parallélisme 25

8.2 Les trois sources de parallélisme 25

8.3 Exemples de calcul 26

9 Méthodologie de parallélisme de tâches 28

9.1 Notions de système de tâches 28

9.2 Conditions de Bernstein 28

9.3 Relation de précédence $\ll$ 29

9.4 Graphe de précédence 29

10 Programmation parallèle avec OpenMP 30

10.1 Modèle fork-join 30

10.2 Mise en œuvre 30

10.3 Statut des variables 31

10.4 Synchronisation 31

10.5 Boucles, réduction, ordonnancement, sections 31

11 Fiche formules & questions de révision 33

11.1 Questions de révision 33

1 Introduction au calcul parallèle

Définition

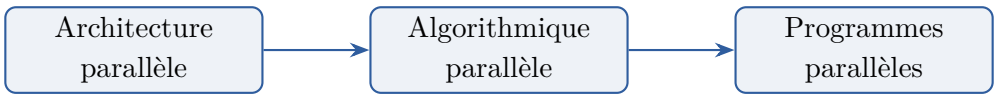
Le **calcul parallèle** consiste à utiliser **plusieurs unités de calcul** (processeurs autonomes) capables de **communiquer** et de **coopérer** dans le but d'**accélérer** la résolution d'**un seul problème**.
Autres noms : calcul parallèle et distribué, calcul à haute performance (HPC), *parallel computing*.

	Algorithme séquentiel	**Algorithme parallèle**
Principe	Suite d'instructions exécutées l'une après l'autre	Le même algorithme exécuté par plusieurs unités de calcul
Dépendance	Chaque instruction dépend de la précédente	Nécessite communication, coopération, synchronisation
Complexité	Somme des opérations	Max des branches exécutées en parallèle

— On donne à chaque processeur une tâche ; ils échangent des données. Une tâche ne doit pas casser celle d'un autre : il faut **ordonnancer** les tâches (rôle du système d'exploitation).

— Question clé : les processeurs partagent-ils la même mémoire ou passent-ils par un réseau ? ⇒ **avant d'écrire l'algorithme, il faut connaître l'architecture.**

— Chaque problème a un **potentiel de parallélisme** (nul ou fort).



1.1 Pourquoi le parallélisme est devenu indispensable

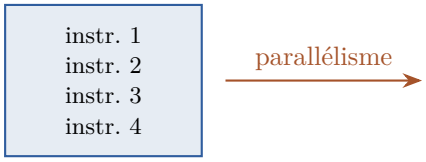
— Les applications « **3T** » demandent 1 Téraflops, 1 Téraoctet de mémoire centrale et 1 Téraoctet/s de bande passante. On parle maintenant d'applications « **3P** » (péta = $10^{15} = 10^3$ téra). *flops* = opérations sur les réels par seconde.

— Elles ne sont réalisables qu'avec un **parallélisme massif** ⇒ HPC (classement mondial : top500.org).

— Grands challenges : modélisation et simulation (aéronautique, météo), ingénierie (génie nucléaire), calcul symbolique (reconnaissance de la parole, langage naturel, vision par ordinateur).

1.2 Temps séquentiel vs temps parallèle

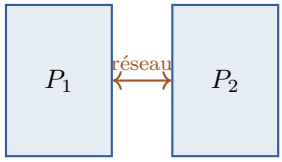
Programme séquentiel



$T_s = T_{\text{traitement}}$

1 facteur : le processeur

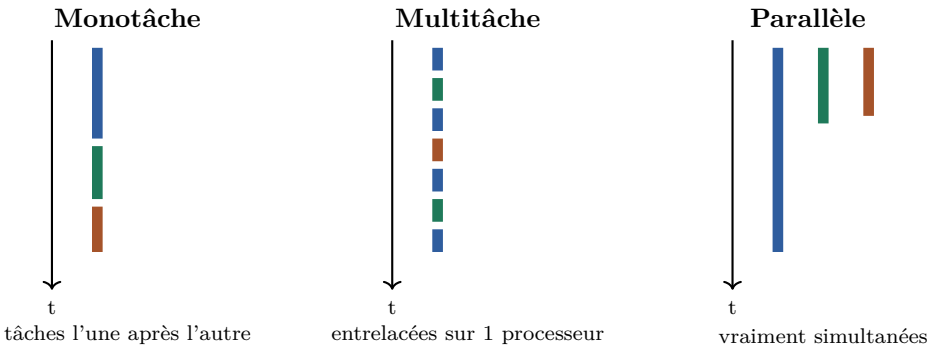
Programme parallèle



$T_p = T_{\text{traitement}} + T_{\text{communication}}$

2 facteurs : processeurs + réseau d'interconnexion

2 Architectures parallèles



Deux grandes catégories : machines **à mémoire partagée** (dont vectorielles) et **à mémoire distribuée**. Le domaine s'est fortement développé depuis 1985. Aujourd'hui, les **clusters** dominent.

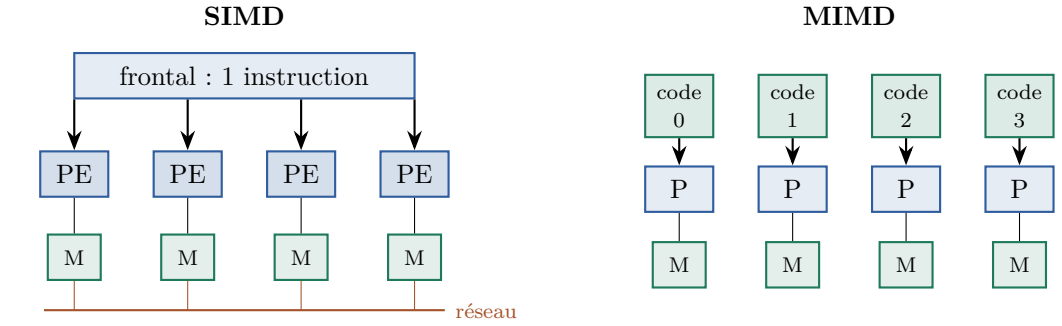
2.1 Classification de Flynn (1966)

Deux flux : **flux d'instructions** (séquence d'instructions exécutées) et **flux de données** (séquence de données appelées par les instructions).

	1 flux de données (SD)	**Plusieurs flux de données (MD)**
1 flux d'instr. (SI)	**SISD** machines séquentielles	**SIMD** machines vectorielles $a_i = b_i + c_i$ pour tout i
Plusieurs flux d'instr. (MI)	**MISD** *aucune machine réelle*	**MIMD** multiprocesseurs

SPMD (Single Program Multiple Data) : variante de MIMD, même programme sur des données différentes. Ordre : **SIMD** < **SPMD** < **MIMD**. Flynn reste simpliste.

2.2 SIMD et MIMD



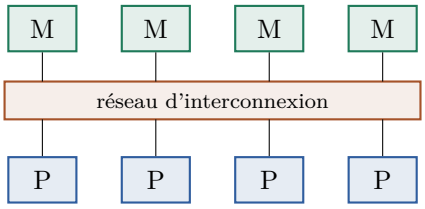
même opération au même moment, synchronisés

chacun son code, asynchrones

	SIMD	MIMD
Principe	Un frontal + processeurs identiques synchronisés : même opération, données différentes	Chaque processeur exécute son propre code sur ses données
Exécution	Synchrone	Asynchrone, indépendante
Processeurs	Spécifiques (petite mémoire locale + réseau)	Ceux du marché
Usage	Calcul régulier (traitement d'images)	Usage général — classe la plus répandue
Exemples	Cray-1, NEC SX-5, CM-1 — en voie de disparition	Mémoire partagée ou distribuée

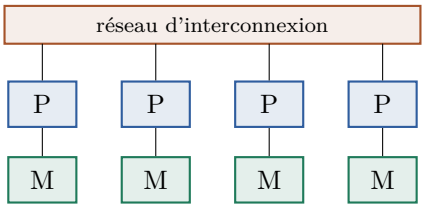
2.3 Mémoire partagée vs mémoire distribuée

Mémoire partagée (SMP)



tous les P accèdent aux mêmes M

Mémoire distribuée

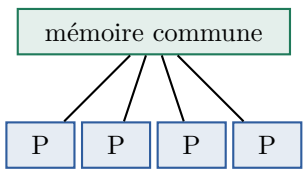


chaque P a sa propre M (messages)

	Mémoire partagée (SMP)	Mémoire distribuée
Mémoire globale	Oui, partagée par tous (2 à 64 processeurs)	Non : chaque processeur a la sienne
Communication	Implicite, via la mémoire	Explicite, par messages sur le réseau
Programmation	Facile	Plus difficile
Extensibilité	Faible, coût élevé	Forte — seules à offrir de très hautes performances
Exemples	PC bi-processeur, Cray SV1, SGI Origin3000	Transputers, Cray T3E, IBM SP3

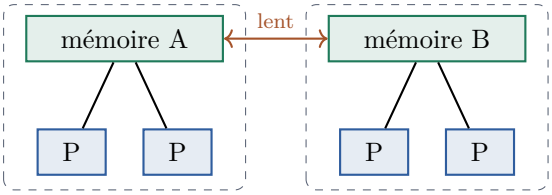
UMA et NUMA

UMA



même temps d'accès pour tous

NUMA (cc-NUMA)



plusieurs SMP reliés : accès local rapide, distant lent

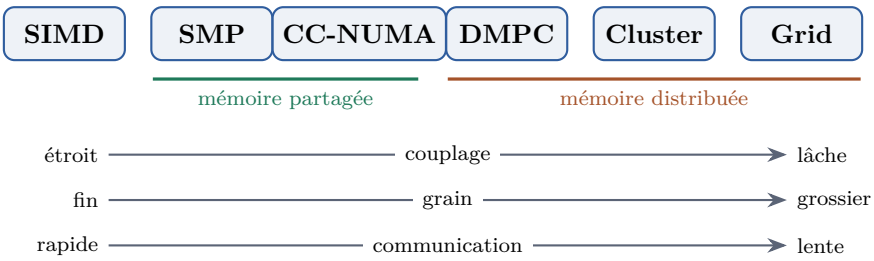
— **CLUMPS** (Clusters of SMP) : mémoire partagée dans chaque SMP, distribuée entre les SMP (ex. IBM SP3, nœuds de 16 processeurs).

— **Hybride** : mémoire physiquement distribuée, programmée comme partagée (ex. SGI Origin2000).

Attention — p processeurs $\neq$ p fois plus rapide

Les processeurs doivent échanger des données ; le coût des communications est très élevé. **Optimiser les mouvements de données est la clé** en mémoire distribuée.

2.4 Classement de Leopold (2001)



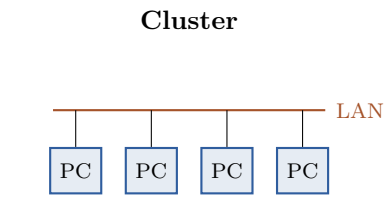
De gauche à droite, les machines s'éloignent (de la puce jusqu'à Internet) : couplage lâche, communication lente ⇒ seulement de gros grains.

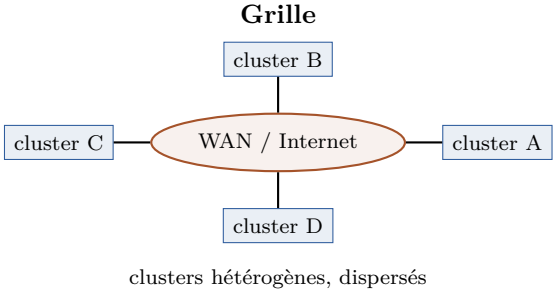
2.5 Système parallèle vs système distribué

Système parallèle	Système distribué
Éléments de calcul qui communiquent et coopèrent pour résoudre rapidement de grands problèmes.	Processeurs autonomes, sans mémoire primaire partagée, qui coopèrent par messages sur un réseau.

Parallèle non distribué : vectorielles, SMP. Parallèle et distribué : stations sur réseau local, CLUMPS. Distribué non parallèle : stations longue distance. Ni l'un ni l'autre : PC isolés.

2.6 Clusters et grilles





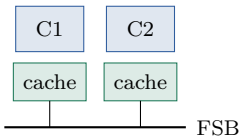
	Cluster (grappe)	Grille (Grid)
Définition	Collection de nœuds **faiblement couplés** reliés par un réseau d'interconnexion	Collection de **clusters hétérogènes**, dispersés géographiquement
Réseau	LAN rapide (Fast Ethernet, Myrinet) : haute bande passante, faible latence	WAN / Internet
Hétérogénéité	Homogène ou hétérogène	Forte ; middleware qui masque l'architecture
Apparition	Années 90 (alternative économique)	Années 2000 — Foster (1998) : comme le réseau électrique
Exemples	Tera-10 (Bull, CEA), TSUBAME (NEC)	GRID5000 (France), TeraGrid (USA)

Époque	1970s	1980s	1990s	2000s	Après 2000
Parallélisme	Processeur	Machine	LAN	WAN	Puce
Architecture	Vectorielle	SMP / MPP	Cluster	Grid	Multi-cœur
Programmation	Vector Fortran	C*, C-Linda, Occam	PVM, MPI, HPF	MPI, OpenMP	?

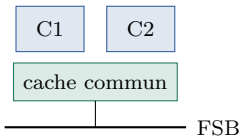
3 Multi-cœurs, GPU et CUDA

3.1 Processeurs multi-cœurs

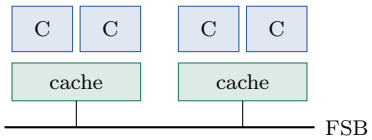
Au moins **deux cœurs** sur la même puce : on augmente la puissance **sans augmenter la fréquence**, avec moins d'énergie et de chaleur que les multiprocesseurs.



1. caches distincts



2. cache commun

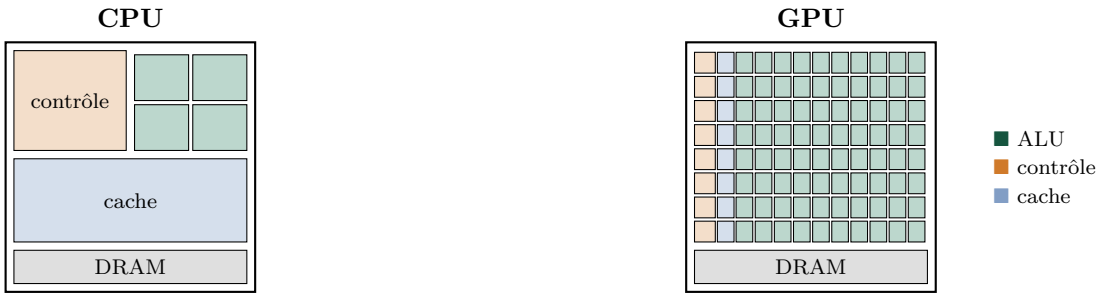


3. « Quad-Core » Intel = 2 puces double-cœur

> **Attention**
>
> Les programmes doivent être repensés, sinon un seul cœur travaille.

3.2 GPU

Circuit de la carte graphique, structure **hautement parallèle**. Libère le CPU des calculs d'affichage.



	CPU	GPU
Transistors	Caches, contrôle	Beaucoup plus d'ALU
Bande passante mémoire	Plus faible	Plus élevée
Rôle	Parties séquentielles	Calcul intensif

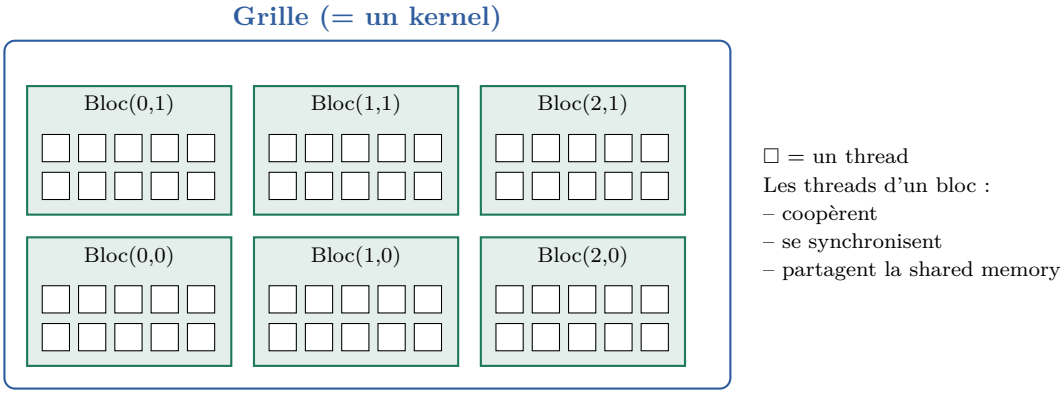
Mémoire graphique partagée (part de la RAM) ou dédiée (sur la carte). Carte dédiée = la plus puissante ; IGP intégré = moins cher, plus économe.

3.3 CUDA (NVIDIA)

— API, extension du langage C : interface de programmation + runtime + bibliothèque mathématique.

— Programme = **code série sur le CPU (host) + kernels sur le GPU (device)** exécutés par des threads.

— Un kernel s'exécute comme une **grille de blocs de threads**. Les threads d'un bloc coopèrent, se synchronisent et partagent la **shared memory**. Les blocs s'exécutent comme des **CTA** sur les cœurs **SM**.



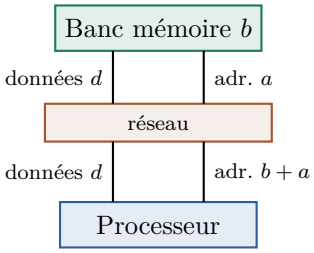
4 Réseaux d'interconnexion

Critères de classement : mode d'opération (synchrone / asynchrone), stratégie de contrôle (centralisée / décentralisée), commutation (circuit / paquet), topologie (**statique** / **dynamique** : connexions établies à la volée).

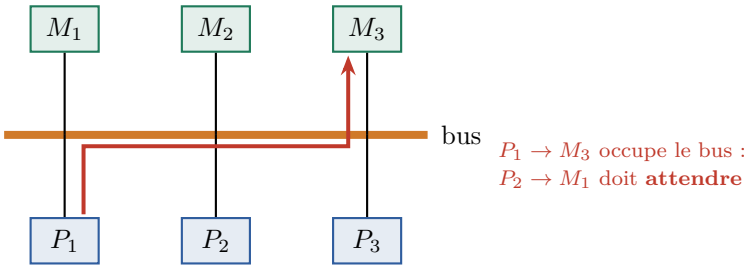
4.1 Machines à mémoire partagée (réseaux dynamiques)

Le réseau relie les **bus de données** et les **bus d'adresses** des processeurs à n'importe quel module mémoire.

- Tous les bus de données ont la même largeur d.
- Une adresse est un couple (b, a) : b = numéro du banc mémoire, a = case dans ce banc.
- Côté processeur, l'adresse a une largeur $b + a$; le réseau utilise b pour choisir le banc, qui ne reçoit que a.

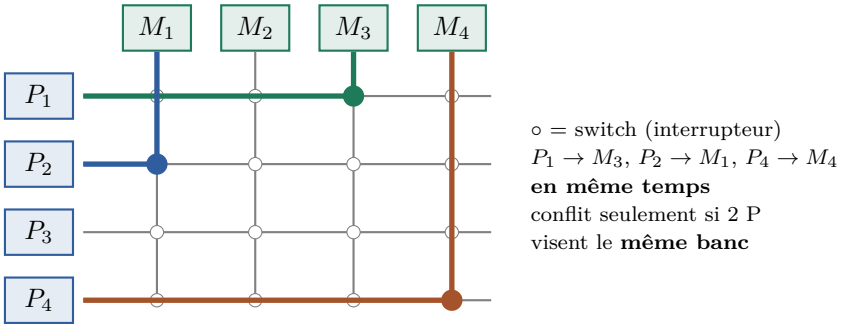


1. Bus (zéro étage)



Un seul banc accessible à la fois : le bus est réservé jusqu'à l'arrivée de l'information. **Aucun parallélisme** ; conflit résolu par exclusion mutuelle.

2. Crossbar (un étage)



Chaque processeur est relié à chaque banc (graphe complet). Les processeurs peuvent envoyer vers des bancs différents **en même temps** ; exclusion mutuelle uniquement sur un même banc. Très coûteux : $p \times m$ switchs.

3. Réseau multi-étages et boîtes d'échange

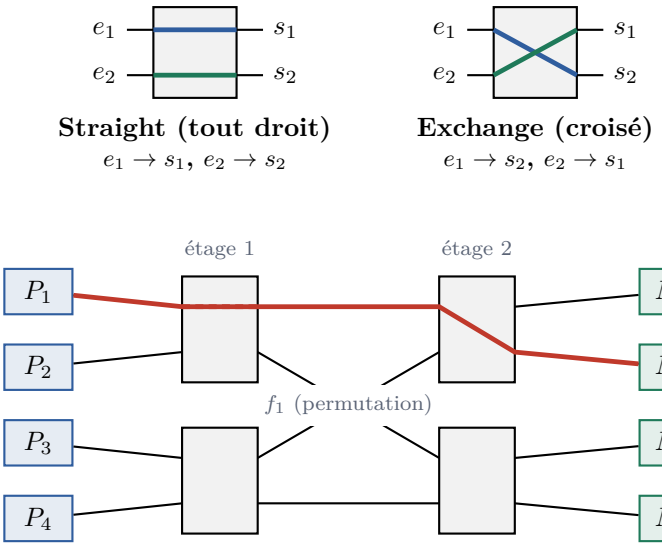


Figure 1 – Réseau multi-étages 4×4 (2 étages de 2 boîtes). En rouge, $P_1 \to M_2$: boîte de l'étage 1 en *straight*, boîte de l'étage 2 en *exchange*. Il n'y a pas de lignes directes : on passe par des boîtes.

À retenir — règle des conflits

Si deux processeurs passent par la même boîte et demandent **la même position** (straight ou exchange), il n'y a pas de conflit : accès simultané. S'ils demandent des positions **différentes**, il faut un **arbitrage**.

	Bus (0 étage)	Crossbar (1 étage)	Multi-étages
Mémoire	Un seul banc à la fois	Plusieurs bancs	Plusieurs bancs
Parallélisme	Aucun	Bancs différents en parallèle	Simultané sans conflit de boîte
Conflit	Exclusion mutuelle	Exclusion mutuelle sur un banc	Arbitrage dans la boîte
Coût	Faible	Élevé ($p \times m$ switchs)	Intermédiaire

4.2 Machines à mémoire distribuée (réseaux statiques)

Le réseau achemine des **messages** entre processeurs (machine à passage de messages).

Entête : destinataire, expéditeur, informations	Corps (données)

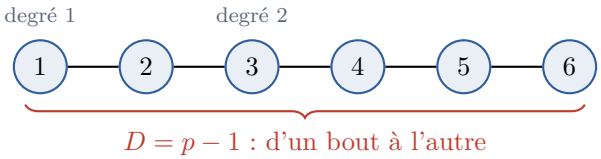
Le réseau est un graphe non orienté $G(X, U)$: sommets = processeurs, arêtes = liens.

Symbole	Nom	Signification
$n = \|X\|$	Ordre	Nombre de processeurs
$m = \|U\|$	Nombre d'arêtes	Nombre de liens
Δ	Degré maximal	Plus grand nombre de liens qui touchent un même nœud
D	Diamètre	Plus grande distance entre deux nœuds (distance = longueur de la plus courte chaîne)
K	Connectivité	Nombre de chemins différents entre deux nœuds

À retenir — deux paramètres pour juger un réseau

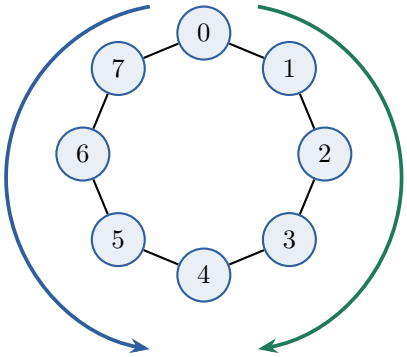
Faisabilité (coût) : $m \searrow$ et $\Delta \searrow \Rightarrow$ coût $\searrow$ (moins de câbles, moins de ports par nœud).
Performance : $D \searrow$ (messages en peu de sauts) et $K \nearrow$ (communications parallèles, tolérance aux pannes) $\Rightarrow$ performance $\nearrow$.
$K = 1 \Rightarrow$ **aucun parallélisme possible**.

Chaîne linéaire (grille 1D)



$n = p$, $m = p - 1$, $D = p - 1$, $\Delta = 2$, $K = 1$ (un seul chemin $\Rightarrow$ pas de parallélisme).

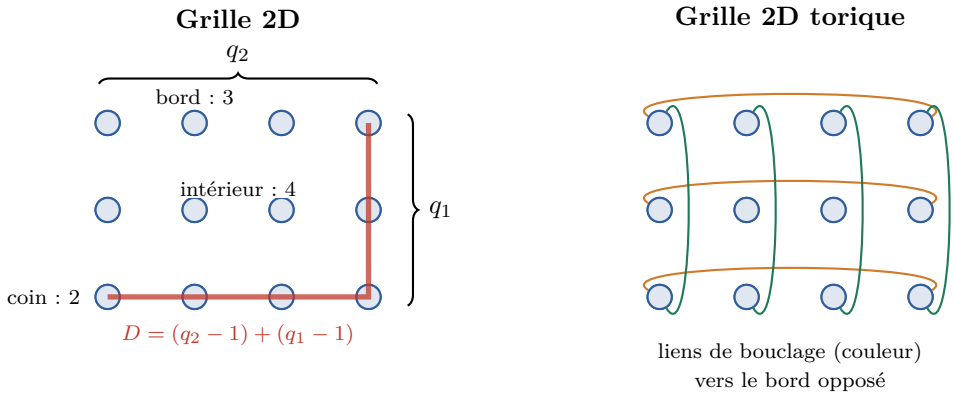
Anneau (réseau torique 1D)



Deux sens possibles : $K = 2$
Au plus un demi-tour : $D = E(p/2)$
La chaîne + un lien qui la referme : $m = p$

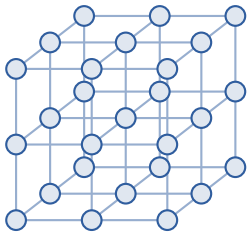
$n = p$, $m = p$, $D = E(p/2)$, $\Delta = 2$, $K = 2$ (parallélisme possible). Le diamètre est divisé par 2 par rapport à la chaîne.

Grille 2D ($q_1 \times q_2$) et grille 2D torique



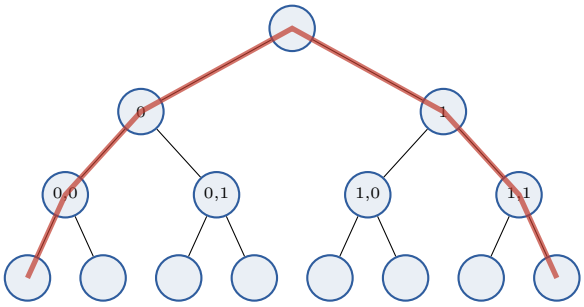
Grille 2D : $n = q_1 q_2$, $m = q_1(q_2 - 1) + q_2(q_1 - 1) = 2q_1q_2 - (q_1 + q_2)$, $D = q_1 + q_2 - 2$, $\Delta = 4$, $K \geq 2$. Chaque nœud est repéré par un couple de coordonnées.
Grille 2D torique : $n = q_1 q_2$, $m = 2q_1q_2$ (4 voisins pour tous : $4n/2$), $D = E(q_1/2) + E(q_2/2)$, $\Delta = 4$, $K \geq 2$.

Grille 3D (maille)



Nœuds repérés par un triplet (x, y, z)
Jusqu'à 6 voisins par nœud
$K \geq 3$

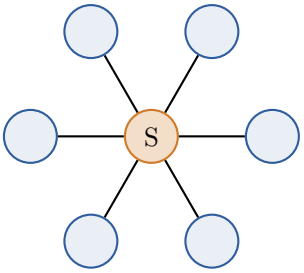
Arbre binaire (h niveaux)



Plus long trajet (rouge) :
feuille → racine → feuille
$(h - 1) + (h - 1)$ sauts

$n = 2^0 + 2^1 + \cdots + 2^{h-1} = 2^h - 1$, $m = 2^h - 2 = n - 1$, $\Delta = 3$ (1 parent + 2 enfants), $D = 2(h - 1) \approx 2 \log_2 n$, $K = 1$ (chemin unique).

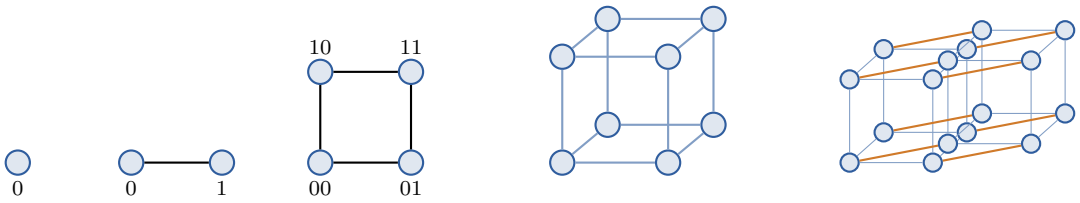
Réseau en étoile



$\Delta = n - 1$ pour le centre $\Rightarrow$ **infaisable** à grande échelle
Modèle client-serveur, centralisé
$m = n - 1, \quad D = 2, \quad K = 1$

Hypercube de dimension d

Construction par récurrence : un 0-cube est un seul processeur ; un d-cube = deux $(d-1)$-cubes superposés, chaque sommet relié à son correspondant.



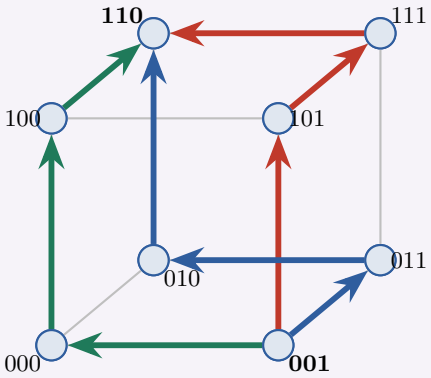
$d = 0$ **: 1** $d = 1$ **: 2** $d = 2$ **: 4** $d = 3$ **: 8** $d = 4$ **: 16**

Grandeur	Récurrence	Résultat
Nœuds	$n_d = 2\, n_{d-1}$	$n = 2^d$
Liens	$m_d = 2\, m_{d-1} + n_{d-1} = 2\, m_{d-1} + 2^{d-1}$	$m = d \cdot 2^{d-1}$
Degré	$\Delta_d = \Delta_{d-1} + 1$	$\Delta = d$
Diamètre	$D_d = D_{d-1} + 1$	$D = d$

Dépliage de $m_d = 2m_{d-1} + 2^{d-1}$ avec $m_0 = 0$: en multipliant la ligne k par 2^{d-k} et en sommant, chacun des d niveaux apporte $2^{d-1} \Rightarrow m_d = d \cdot 2^{d-1}$. Vérification : 2^d nœuds de degré d, chaque lien compté deux fois : $d \cdot 2^d/2$.

Numérotation (code de Hamming) : chaque nœud reçoit un numéro binaire sur d bits ; deux nœuds sont **voisins s'ils diffèrent d'un seul bit** ; la distance = **distance de Hamming** (nombre de bits différents).

Exemple — chemins parallèles de 001 à 110 dans le 3-cube



001 et 110 diffèrent sur 3 bits ⇒ distance 3.
On change un bit par saut ; on choisit le premier :

bit gauche d'abord : 001 → 101 → 111 → 110
bit du milieu d'abord : 001 → 011 → 010 → 110
bit droit d'abord : 001 → 000 → 100 → 110

3 chemins sans nœud intermédiaire commun.

4.3 Tableau récapitulatif des topologies

Topologie	n	m	D	Δ	K
Chaîne linéaire	p	$p-1$	$p-1$	2	1
Anneau	p	p	$E(p/2)$	2	2
Grille 2D $q_1 \times q_2$	$q_1 q_2$	$2q_1q_2-(q_1+q_2)$	q_1+q_2-2	4	≥ 2
Grille 2D torique	$q_1 q_2$	$2q_1q_2$	$E(q_1/2)+E(q_2/2)$	4	≥ 2
Grille 3D	—	—	—	6	≥ 3
Arbre binaire (h niveaux)	2^h-1	2^h-2	$2(h-1)$	3	1
Étoile	n	$n-1$	2	$n-1$	1
Hypercube (dim. d)	2^d	$d \cdot 2^{d-1}$	d	d	d

E = partie entière. Hypercube : $K = d$ se déduit de l'exemple du 3-cube (3 chemins). Grille 3D : le cours ne donne que $K \geq 3$.

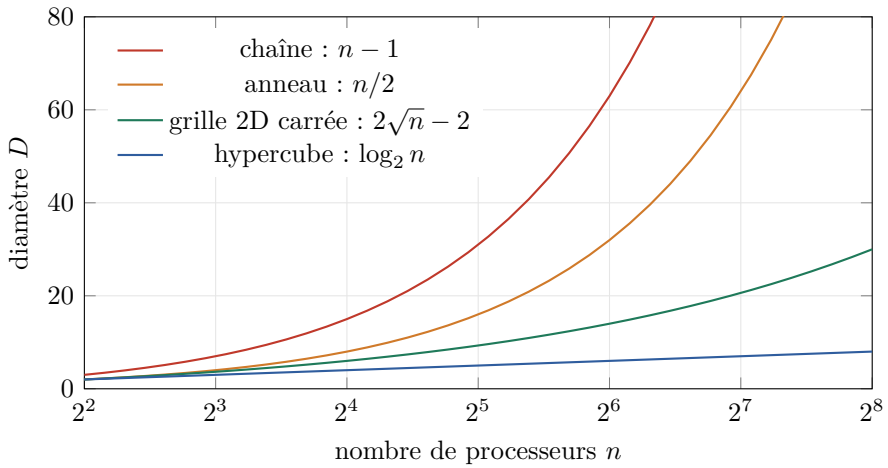


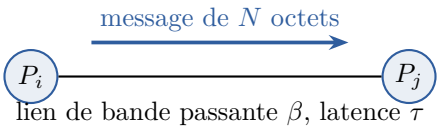
Figure 2 – Le diamètre (nombre de sauts au pire) croît beaucoup moins vite pour la grille et l'hypercube.

En fonction de n	**Linéaire**		**Anneau**		**Grille 2D carrée** ($q = \sqrt{n}$)
m (coût)	$n-1$	$\nearrow$	n	$\nearrow$	$\approx 2n$
Δ (coût)	2	$=$	2	$\nearrow$	4
D (performance)	$n-1$	$\searrow$	$n/2$	$\searrow$	$\approx 2\sqrt{n}$
K	1		2		≥ 2

On paie un peu plus de liens (coût $\nearrow$) pour un diamètre bien plus petit (performance $\nearrow$).

5 Communications dans un réseau à mémoire distribuée

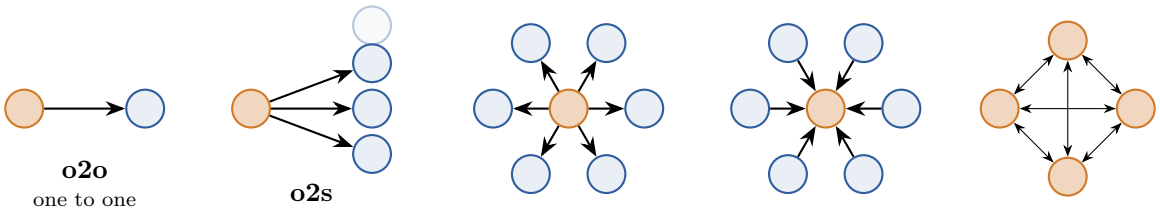
5.1 Temps de communication



$$T_{\text{comm}}\ (1 \text{ lien}) = \tau + \frac{N}{\beta} \qquad \text{sur un chemin de } L \text{ liens : } L \cdot \left(\tau + \frac{N}{\beta}\right)$$

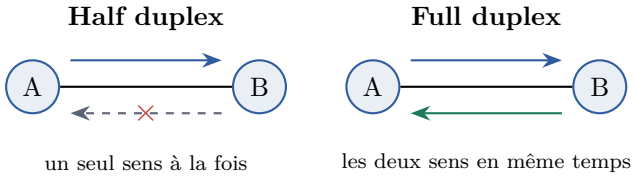
N : taille du message (octets) ; β : bande passante du lien (octets/s) ; τ : latence = temps d'envoi d'un message vide (s).

5.2 Types de communication

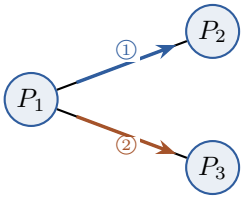


Source \ destination	One	Some	All
One	o2o	o2s	o2a
Some	s2o	s2s	s2a
All	a2o	a2s	a2a

5.3 Types de liens et dispositif de communication

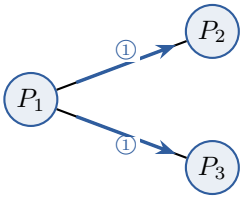


Processor-bound



un seul envoi à la fois
depuis un processeur : étape 1 puis 2

Link-bound

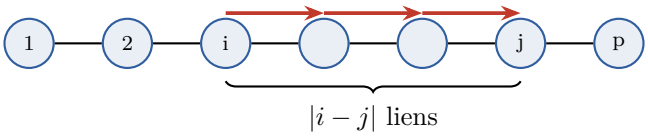


envois simultanés
sur tous les liens : étape 1 partout

Problématique : calculer le temps de communication pour chaque réseau et chaque type de communication.

5.4 Réseau linéaire (p processeurs)

o2o : $i \to j$



$$T = |i-j| \cdot \left(\tau + \frac{N}{\beta}\right)$$

o2a : $i \to$ tous

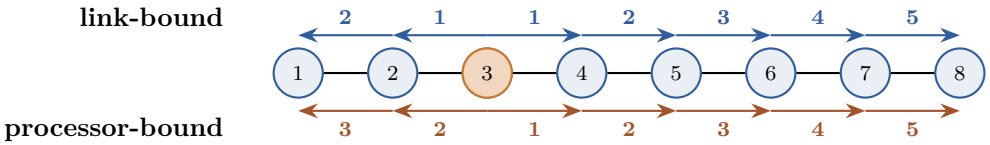


Figure 3 – $p = 8$, source $i = 3$ (orange). Les nombres = étape à laquelle le lien est utilisé. Le côté le plus éloigné (5 liens) impose 5 étapes dans les deux modes ; en processor-bound, on sert d'abord le côté le plus long.

— **Cas 1 — link-bound** : envoi des deux côtés en parallèle ; le temps est dicté par le côté le plus éloigné :

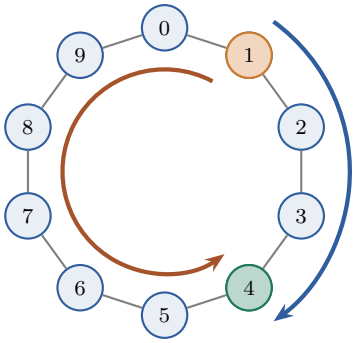
$$T = \max(i-1,\ p-i) \cdot \left(\tau + \frac{N}{\beta}\right)$$

— **Cas 2 — processor-bound** : un processeur ne peut pas faire deux envois en même temps. C'est le même temps, sauf quand les deux côtés sont à égale distance et ne peuvent pas être servis simultanément. Cas noté dans le cours : $i = p/2$ (p pair), $T = \frac{p}{2} \cdot \left(\tau + \frac{N}{\beta}\right)$.

5.5 Anneau (p processeurs)

Entre i et j, deux chemins : longueur $|i-j|$ et longueur $p - |i-j|$.

o2o : $i \to j$



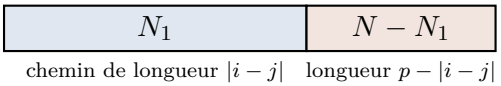
chemin court : $|i-j| = 3$ liens
porte le plus gros morceau N_1

chemin long : $p - |i-j| = 7$ liens
porte le reste $N - N_1$

— **Cas 1 — un seul chemin (le plus court)** :

$$T = \min\left(|i-j|,\ p-|i-j|\right) \cdot \left(\tau + \frac{N}{\beta}\right)$$

— **Cas 2 — sur les deux chemins** : le message est coupé en N_1 et $N - N_1$ (pas forcément égaux) ; le plus gros morceau prend le chemin le plus court ; les deux envois sont simultanés :



$$T = \max\left(|i-j| \cdot \left(\tau + \tfrac{N_1}{\beta}\right),\ (p-|i-j|) \cdot \left(\tau + \tfrac{N-N_1}{\beta}\right)\right)$$

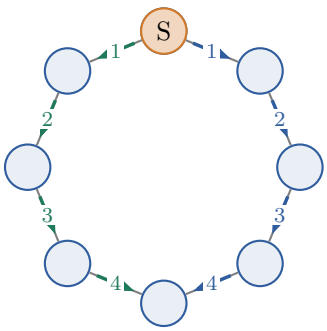
À retenir — anneau, o2o sur deux chemins

— Quand N_1 augmente, le 1er terme augmente et le 2e diminue $\Rightarrow$ T **est minimal quand les deux termes sont égaux**.

— Le gain est vraiment intéressant quand i et j sont **diamétralement opposés** (ex. $i = 1$, $j = p/2$) : les deux chemins ont la même longueur, on coupe le message en deux moitiés égales.

— Link-bound : les deux envois partent en parallèle. Processor-bound : un envoi à la fois, donc un décalage. Dans les deux cas, le temps final est le même.

o2a : $i \to$ tous (link-bound)

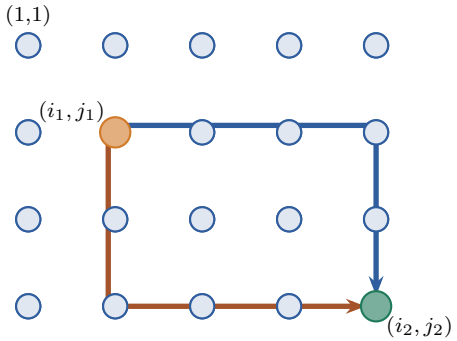


Diffusion dans les deux sens.
$p = 8$: 4 étapes $= E(p/2)$.
Le nœud opposé peut recevoir le message 2 fois.

$$T = E\left(\frac{p}{2}\right) \cdot \left(\tau + \frac{N}{\beta}\right)$$

5.6 Grille 2D ($q_1 \times q_2$)

Nœuds repérés par (ligne, colonne) de $(1,1)$ à (q_1, q_2). Distance de (i_1, j_1) à (i_2, j_2) : $|i_1 - i_2| + |j_1 - j_2|$ (distance de Manhattan).



Distance $= |i_1 - i_2| + |j_1 - j_2| = 3 + 2 = 5$

chemin 1 : d'abord horizontal, puis vertical

chemin 2 : d'abord vertical, puis horizontal

Chacun porte une moitié $N/2$ du message.

— **Cas 1 — un seul chemin** :

$$T = (|i_1 - i_2| + |j_1 - j_2|) \cdot \left(\tau + \frac{N}{\beta}\right)$$

— **Cas 2 — deux chemins** (deux envois simultanés de deux moitiés $N/2$) :

$$T = (|i_1 - i_2| + |j_1 - j_2|) \cdot \left(\tau + \frac{N}{2\beta}\right)$$

— **Cas 3 — plusieurs chemins** : même idée, on découpe le message sur plusieurs chemins disjoints (non détaillé dans le cours).

6 Grain, degré de parallélisme, T_{opt} et P_{opt}

Définition — grain (granularité)

Taille moyenne des tâches élémentaires issues de la parallélisation (en nombre d'instructions ou en temps d'exécution). Son choix est **fortement lié à l'architecture** : communication lente ⇒ **gros grain** ; communication rapide ⇒ **grain fin**.

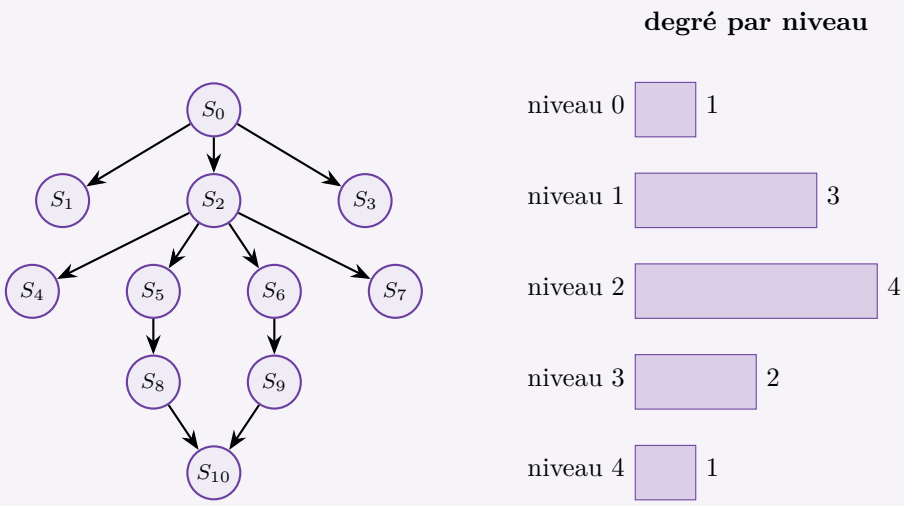
Attention — compromis du grain

Grain trop fin ⇒ surcoût de communication et de synchronisation. Grain trop gros ⇒ déséquilibre de charge, processeurs inactifs.

Définition — degré de parallélisme

Nombre de sous-tâches exécutées **simultanément**. Il indique le nombre de processeurs utilisables et varie pendant l'exécution ⇒ on considère D_{max}, D_{moy}, D_{min}, évalués statiquement (compilation) ou dynamiquement (exécution).

Exemple — calcul des degrés



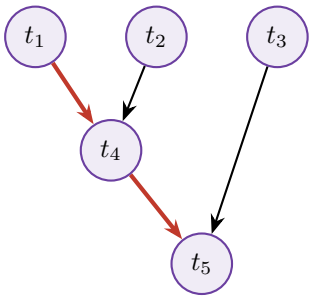
$$D_{max} = 4, \quad D_{min} = 1, \quad D_{moy} = \frac{1+3+4+2+1}{5} = \frac{11}{5} = 2{,}2$$

6.1 Temps et nombre de processeurs optimaux

— $\mathbf{T_{opt}}$: temps optimal d'exécution parallèle = somme des durées des tâches du **chemin critique** (le plus long chemin) du **graphe de dépendance** (DAG). On le détermine **indépendamment du nombre de processeurs**.

— $\mathbf{P_{opt}}$: nombre **minimal** de processeurs permettant d'atteindre T_{opt}. C'est un optimum global : il ne dépend pas de la représentation.

— Toujours : $T_p \geq T_{opt} \Rightarrow S_p \leq T_1/T_{opt}$, quel que soit p.



Chemin critique en rouge :
$t_1 \to t_4 \to t_5$ (3 tâches)
$\Rightarrow T_{opt} = 3$

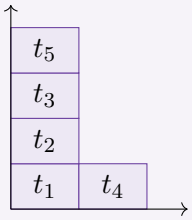
Exemple — exercice 1 — $P_{opt} = D_{max}$

$t_1 : a \leftarrow a + 2$, $t_2 : b \leftarrow b - 4$, $t_3 : d \leftarrow d + 9$, $t_4 : c \leftarrow a \times b - d$ (1 u.t. chacune) $\Rightarrow$ $t_1 \parallel t_2 \parallel t_3 \Rightarrow t_4$.
$D_{min} = 1$, $D_{max} = 3$, $D_{moy} = 2$, $\quad \mathbf{T_{opt} = 2}$, $\mathbf{P_{opt} = 3 = D_{max}}$.

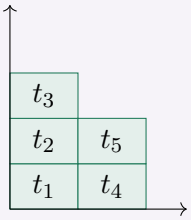
Exemple — exercice 2 — $P_{opt} < D_{max}$

On ajoute $t_5 : e \leftarrow e + 1$ (indépendante) : $t_1 \parallel t_2 \parallel t_3 \parallel t_5 \Rightarrow t_4$.

naïf : $p = 4$



optimal : $p = 3$



$D_{max} = 4$ mais t_5 peut passer à côté de t_4 : $T_{opt} = 2$ avec 3 processeurs $\Rightarrow \mathbf{P_{opt} = 3 < D_{max} = 4}$.

7 Évaluation des performances

Notations : n = taille du problème, $T_1(n)$ = temps séquentiel, $T_p(n)$ = temps parallèle sur p processeurs.

Accélération (speedup)

$$S_p(n) = \frac{T_1(n)}{T_p(n)}$$

- $S_p < 1$: ralentissement
- $1 < S_p \leq p$: cas normal
- $S_p > p$: **super-linéaire**

Efficacité

$$E_p(n) = \frac{S_p(n)}{p} = \frac{T_1(n)}{pT_p(n)}$$

Taux d'utilisation des processeurs. $E_p = 1$ = idéal. Maximale quand la charge est **équilibrée**.

À retenir — pourquoi une accélération super-linéaire ?

T_1 = temps de swap (mémoire/cache) + temps de traitement. Si les données ne tiennent pas dans le cache d'un seul processeur (ex. matrice $10^6 \times 10^6$), il y a beaucoup de swaps. En parallèle, chaque processeur ne manipule qu'une partie qui tient en cache : le swap disparaît et T_p = traitement$/p < T_1/p \Rightarrow S_p > p$.

7.1 Loi d'Amdahl

Si une fraction f du programme est strictement séquentielle :

$$S_p \leq \frac{1}{f + \dfrac{1-f}{p}} \xrightarrow[p\to\infty]{} \frac{1}{f}$$

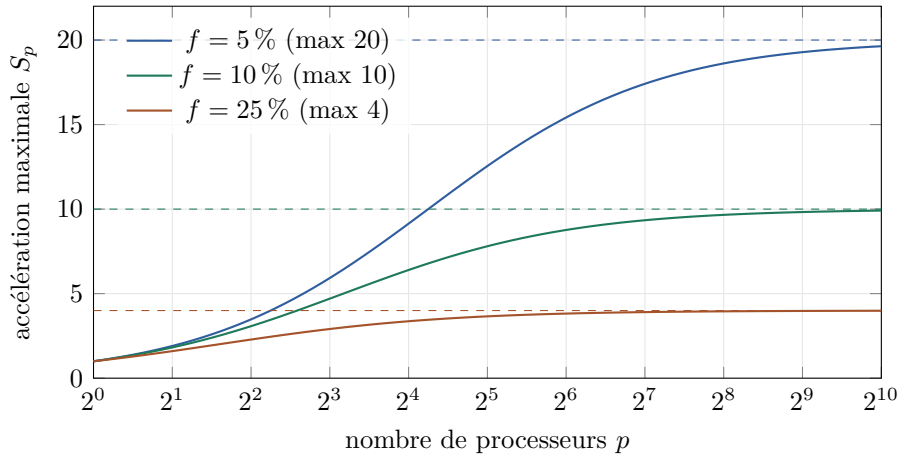


Figure 4 – Loi d'Amdahl : la partie séquentielle plafonne l'accélération à $1/f$, quel que soit p.

Ex. : $f = 10\,\% \Rightarrow$ l'accélération ne dépassera **jamais 10**.

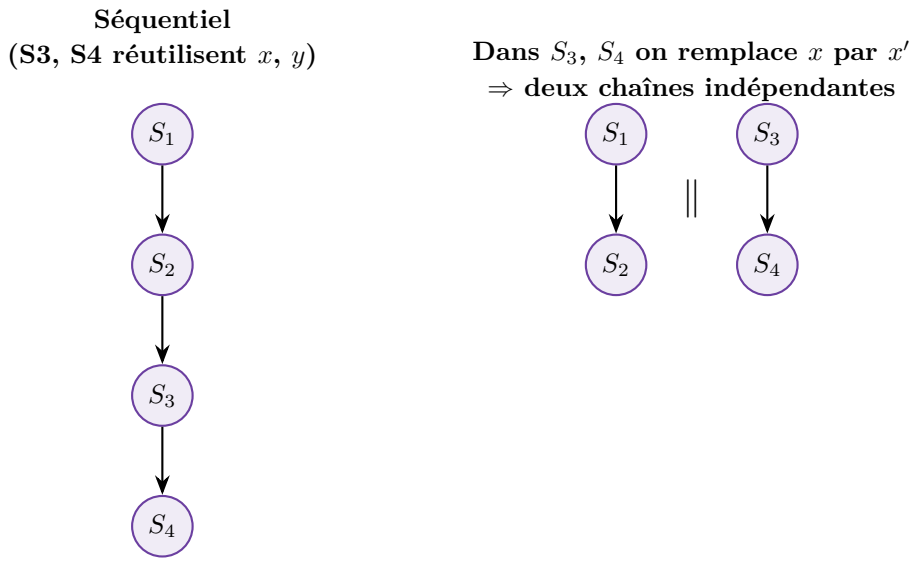
8 Formes et sources de parallélisme

8.1 Formes de parallélisme

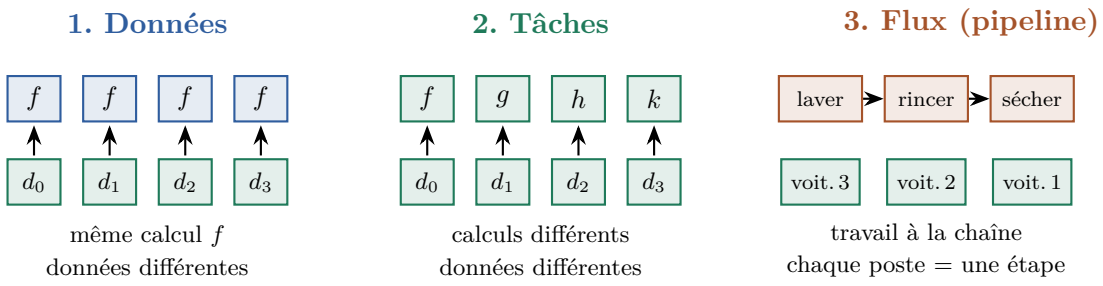
Implicite	Extrait automatiquement par le compilateur ou le matériel (pipeline, superscalaire). Le programmeur n'intervient pas.
Explicite	Le développeur parallélise lui-même (threads, MPI, OpenMP…).

Deux approches : **séquentiel → parallèle** (adapter un programme existant) ou **parallèle dès la conception** (facile sur du code simple, difficile sur du code complexe).

> **Attention — quand peut-on paralléliser ?**
> Si S_1 et S_2 sont **dépendantes** (accès à la même case mémoire : $x := f(\dots)$; $y := x + \dots$), elles s'exécutent en **séquentiel**.



8.2 Les trois sources de parallélisme



Source	Principe	Caractéristiques
Données	Le **même calcul** appliqué en parallèle à des **données différentes** ; exploite la régularité (tableaux, listes, matrices).	Travail répétitif, faible complexité, facile à découper. Difficulté : distribuer les données.
Tâches	Des **calculs différents** sur des **données différentes**, en même temps.	On ne répartit pas au hasard : on étudie les dépendances. But : minimiser temps et ressources.
Flux	Travail **à la chaîne** : chaque processeur est spécialiste d'une étape.	Moins courant. Exploite la régularité des données.

Combiner plusieurs modes = **parallélisme mixte**.

8.3 Exemples de calcul

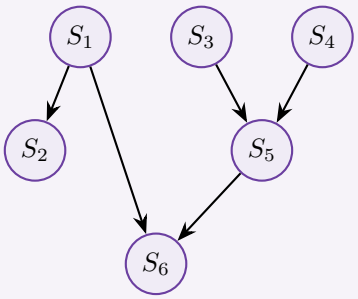
Données. `pour i de 1 à n : A(i) <- A(i) + 4`. Cas idéal $p = n$: $T_1 = n$, $T_n = 1 \Rightarrow S_n = n$, $E_n = 1$.

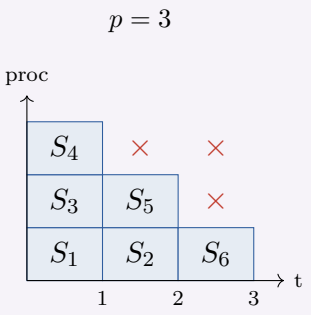
Tâches (1 u.t. par tâche). $t_1 : a \leftarrow a + 2$, $t_2 : b \leftarrow b - 4$, $t_3 : d \leftarrow d + 9$, $t_4 : c \leftarrow a \times b - d \Rightarrow t_1 \parallel t_2 \parallel t_3 \Rightarrow t_4$.
$T_1 = 4$, avec $p = 3$: $T_3 = 2 \Rightarrow S_3 = 2$, $E_3 = 2/3 \approx 66\,\%$.

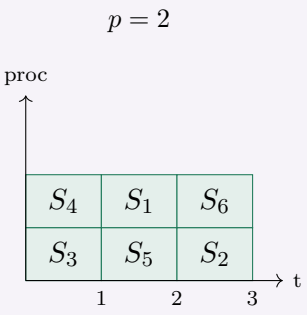
Exemple — graphe de dépendance + diagramme de Gantt (cours)

$S_1 : a = 2$ $S_2 : b = a \times 2$ $S_3 : c = 4$ $S_4 : d = 5$ $S_5 : e = c - d$ $S_6 : f = a + e$

Graphe de dépendance



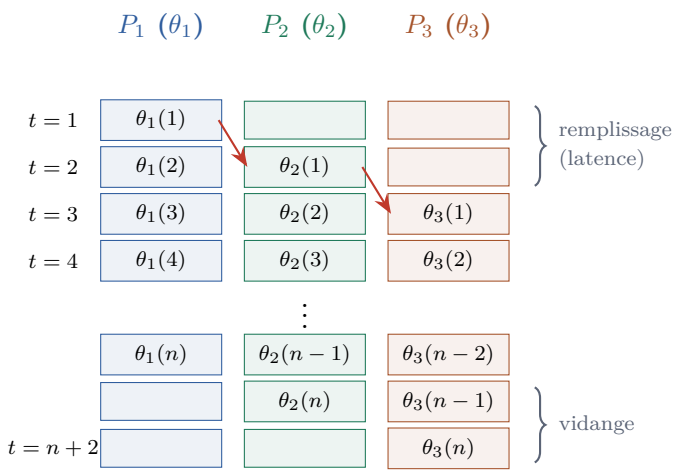




3 processeurs : $T_{seq} = 6$, $T_3 = 3 \Rightarrow S_3 = 2$, $E_3 = 2/3$ (66 %) — × = processeur inactif.
2 processeurs : $T_2 = 3 \Rightarrow S_2 = 2$, $E_2 = 1$ (100 %). Même accélération avec moins de processeurs ⇒ meilleure efficacité.

Parallélisme de flux (pipeline)

`pour i de 1 à n : C(i) <- [(A(i)+2) * B(i)] / 3` → 3 étapes θ_1 (+), θ_2 (×), θ_3 (/) sur P_1, P_2, P_3.



	T_1	T_p	S_p	E_p
3 étapes	$3n$	$n+2$	$\dfrac{3n}{n+2} \to 3$	$\dfrac{n}{n+2} \to 1$
p **étapes**	$p\,n$	$n+p-1$	$\dfrac{p\,n}{n+p-1} \approx p$ si $p \ll n$	$\dfrac{n}{n+p-1} \approx 1$ si $p \ll n$

Les $p-1$ unités supplémentaires = **latence du pipeline** (remplissage/vidange).

9 Méthodologie de parallélisme de tâches

Chapitre IV — environnement homogène

9.1 Notions de système de tâches

La décomposition d'un programme en tâches **n'est pas unique** : elle dépend de la **granularité** choisie. **On choisit le grain dès le début.**

Exemple — produit matrice × vecteur

```
pour i = 1 à n faire
   pour j = 1 à n faire
      Y(i) = Y(i) + A(i,j) * X(j)
   fin pour
fin pour
```

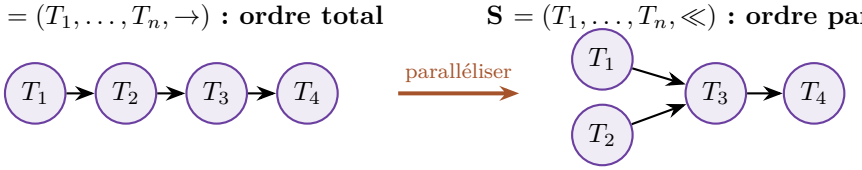
Granularité 1 (la plus grosse) : toute la boucle sur j pour un i
Granularité 2 : une instruction $Y(i) = Y(i) + A(i,j) \cdot X(j)$
Granularité 3 (la plus fine) : une opération $+$ ou $\times$

Définition — tâche

Unité **indivisible** définie par le quadruplet $(E_i, S_i, F_i, \theta_i)$: E_i = entrées, S_i = sorties, F_i = opération, θ_i = coût.

- Une tâche s'exécute sur **un seul processeur**.
- Son exécution est **non préemptive** (on ne l'interrompt pas).
- Modèle **statique** : $(E_i, S_i, F_i, \theta_i)$ connus à l'avance.

PS $= (T_1, \ldots, T_n, \rightarrow)$ **: ordre total** **S** $= (T_1, \ldots, T_n, \ll)$ **: ordre partiel**

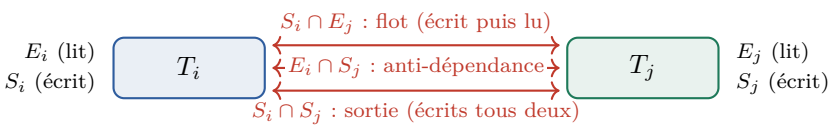


Un bloc ayant trop de dépendances peut être considéré comme **une seule tâche** lors du choix de la granularité. On a intérêt à rendre la relation d'ordre la plus partielle possible.

9.2 Conditions de Bernstein

Deux tâches T_i et T_j sont **indépendantes** (non interférentes) si :

$$S_i \cap S_j \;=\; S_i \cap E_j \;=\; E_i \cap S_j \;=\; \varnothing$$



Autrement dit, aucune variable n'est écrite par l'une et lue ou écrite par l'autre.

Attention — remarque du cours

Si t_1 et t_2 **ne sont pas indépendantes** $\Rightarrow$ elles **ne peuvent pas** s'exécuter en parallèle. Le cours précise : condition nécessaire mais non suffisante, « l'inverse n'est pas correct ».

9.3 Relation de précédence $\ll$

$$T_i \ll T_j \iff T_i \to T_j \quad \text{et} \quad (S_i \cap S_j) \cup (S_i \cap E_j) \cup (E_i \cap S_j) \neq \varnothing$$

$T_i \to T_j$ = **précédence séquentielle** (T_i est avant T_j dans le programme) et les deux tâches sont en conflit sur au moins une variable.

Exemple — transitivité

$t_1 : a = b + c$
$t_2 : e = a - b$
$t_3 : f = d \times e$

$t_1 \ll t_2$ (via a), $t_2 \ll t_3$ (via e)
$\Rightarrow \mathbf{t_1 \ll t_3}$: $\ll$ est **transitive**.

Exemple — l'ordre séquentiel compte

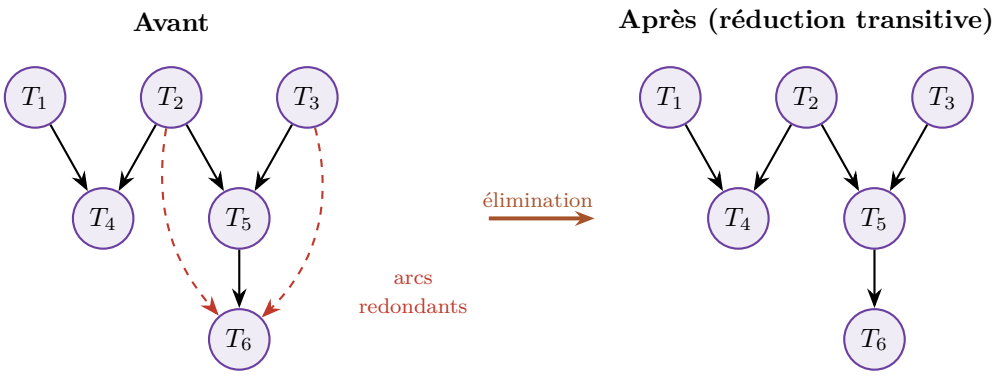
$T_i : A = B + C$
$T_j : B = E - F$

$E_i \cap S_j = \{B\} \neq \varnothing$, mais on ne peut pas conclure $T_i \ll T_j$ ou $T_j \ll T_i$ **sans l'ordre séquentiel** $\to$.

9.4 Graphe de précédence

Graphe de tâches = **graphe orienté sans cycle** : un arc $T_i \to T_j$ si $T_i \ll T_j$.

Exemple : $T_1 \ll T_4$; $T_2 \ll T_4$, $T_2 \ll T_5$, $T_2 \ll T_6$; $T_3 \ll T_5$, $T_3 \ll T_6$; $T_5 \ll T_6$.

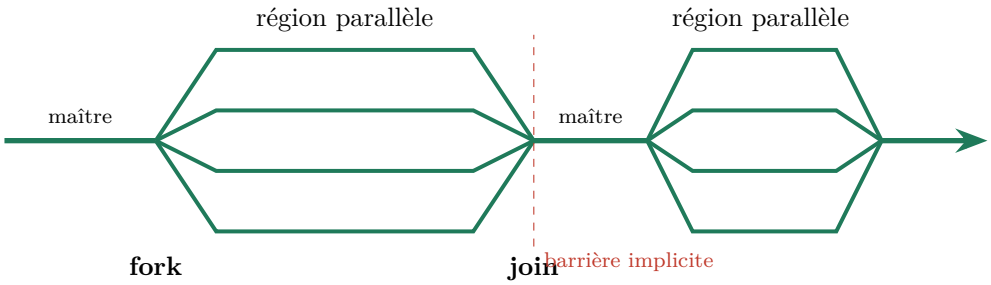


$T_2 \to T_6$ et $T_3 \to T_6$ se déduisent par transitivité ($T_2 \to T_5 \to T_6$, $T_3 \to T_5 \to T_6$) : **on élimine les dépendances redondantes.**

10 Programmation parallèle avec OpenMP

OpenMP (Open MultiProcessing) : API à base de **directives** pour la programmation parallèle en **mémoire partagée** (C, C++, Fortran). Approche la plus simple : paralléliser les boucles.

10.1 Modèle fork-join



— Un seul thread au départ : le **thread maître**.

— **Fork** : début de région parallèle, création d'une équipe de threads.

— **Join** : fin de région, **barrière implicite** ; seul le maître continue.

— Thread = « processus léger » ; les threads d'un processus partagent sa mémoire virtuelle.

10.2 Mise en œuvre

```c
#include <omp.h>
// compilation : gcc -fopenmp -o mon_prog mon_source.c
// nombre de threads : export OMP_NUM_THREADS=2  (defaut = nombre de CPU)
double t0 = omp_get_wtime();
/* travail */
printf("Temps : %f\n", omp_get_wtime() - t0);
```

Fonction	Rôle
`omp_get_num_threads()`	nb de threads
`omp_get_thread_num()`	rang du thread
`omp_set_num_threads(n)`	fixe le nb
`omp_in_parallel()`	1 si région //.

```c
#pragma omp parallel
{
  int rang = omp_get_thread_num();
  printf("Hello du thread %d\n", rang);
  if (rang == 0)
    printf("Total : %d\n",
           omp_get_num_threads());
}
```

10.3 Statut des variables

`shared`	Par défaut, toutes les variables sont partagées (sauf les compteurs de boucle)
`private`	Privée à chaque thread, **non initialisée**
`firstprivate`	Privée, initialisée avec la valeur d'origine
`lastprivate`	Privée ; la variable d'origine reçoit la valeur de la dernière itération
`threadprivate`	Privée, persiste d'une région parallèle à l'autre (initialisée par `copyin`)

Clauses de `parallel` : `num_threads(n)`, `if(expr)` (séquentiel si expr = 0), `default(shared|none)`, `copyin`, `copyprivate` (diffuse après un `single`, incompatible avec `nowait`), `nowait` (supprime la barrière implicite).

10.4 Synchronisation

`master`	Exécuté par le maître (rang 0) seul ; les autres continuent
`single`	Exécuté par un seul thread (le premier arrivé) ; les autres attendent
`barrier`	Tous les threads s'attendent
`critical`	Un seul thread à la fois dans la section
`atomic`	Mise à jour atomique d'une variable (`++`, `-`, op. logique)

10.5 Boucles, réduction, ordonnancement, sections

```
#pragma omp parallel for reduction(+ : somme) schedule(dynamic, 4)
for (i = 0; i < N; i++) somme += f(i);       // i est privee par defaut

#pragma omp parallel for ordered
for (i = 0; i < N; i++) {
  ...
  #pragma omp ordered
  printf("%d\n", i);                          // affiche dans l'ordre
}

#pragma omp sections
{
  #pragma omp section
  { tache_A(); }
  #pragma omp section
  { tache_B(); }
}
```



schedule	Comportement
`static`	Par défaut : N/p itérations par thread ; avec une taille de bloc, distribution circulaire (round-robin)
`dynamic`	Un thread qui termine prend le bloc suivant — adapté aux itérations de coûts inégaux
`guided`	Comme dynamic, mais la taille des blocs décroît : $x = N/p$, $y = (N - x)/p$, …
`runtime / auto`	Choix à l'exécution / laissé au compilateur

Exemple du cours : initialisation d'une matrice 9000×9000 ($a[i][j] = i \cdot j$) avec `parallel for num_threads(2) private(j)` : gain proche de 2, limité par la création des threads et les accès mémoire (`malloc`).

11 Fiche formules & questions de révision

Accélération	$S_p = T_1/T_p$ (normal : $1 < S_p \leq p$; super-linéaire : $S_p > p$)
Efficacité	$E_p = S_p/p = T_1/(p\,T_p)$ (idéal = 1)
Amdahl	$S_p \leq 1/\big(f + (1-f)/p\big) \to 1/f$
Pipeline (p étapes)	$T_1 = pn$, $T_p = n + p - 1$, $S_p \approx p$, $E_p \approx 1$ si $p \ll n$
Degré moyen	D_{moy} = moyenne des degrés de chaque étape
T_{opt} / P_{opt}	T_{opt} = chemin critique du DAG ; P_{opt} = min de processeurs pour T_{opt} ; $P_{opt} \leq D_{max}$
Borne	$T_p \geq T_{opt} \Rightarrow S_p \leq T_1/T_{opt}$
Communication	1 lien : $\tau + N/\beta$; L liens : $L(\tau + N/\beta)$
Chaîne	o2o : $\lvert i-j\rvert \left(\tau + \frac{N}{\beta}\right)$; o2a link-bound : $\max(i-1, p-i) \left(\tau + \frac{N}{\beta}\right)$
Anneau	o2o 1 chemin : $\min(\lvert i-j\rvert, p - \lvert i-j\rvert) \left(\tau + \frac{N}{\beta}\right)$; o2a : $E(p/2) \left(\tau + \frac{N}{\beta}\right)$
Grille 2D	o2o : $(\lvert i_1 - i_2\rvert + \lvert j_1 - j_2\rvert) \left(\tau + \frac{N}{\beta}\right)$; 2 chemins : $(\dots)(\tau + N/2\beta)$
Hypercube	$n = 2^d$, $m = d\,2^{d-1}$, $D = \Delta = d$, distance = Hamming
Bernstein	$S_i \cap S_j = S_i \cap E_j = E_i \cap S_j = \varnothing$
Précédence	$T_i \ll T_j \iff T_i \to T_j$ et $(S_i \cap S_j) \cup (S_i \cap E_j) \cup (E_i \cap S_j) \neq \varnothing$

11.1 Questions de révision

Q1. Quelles sont les 4 classes de Flynn ? Laquelle n'a aucune machine réelle ? — SISD, SIMD, MISD, MIMD ; c'est MISD.

Q2. Quelles sont les trois sources de parallélisme ? — Données, tâches, flux (pipeline). Leur combinaison = parallélisme mixte.

Q3. Différence implicite / explicite ? — Implicite : extrait par le compilateur/matériel. Explicite : écrit par le développeur.

Q4. $t_1 \parallel t_2 \parallel t_3 \Rightarrow t_4$ sur 3 processeurs : S et E ? — $T_1 = 4$, $T_3 = 2$, $S = 2$, $E = 2/3$.

Q5. Pipeline à 3 étages sur $n = 100$ données : S_3 ? — $300/102 \approx 2{,}94$; $E_3 = 100/102 \approx 0{,}98$.

Q6. Pourquoi peut-on avoir $S_p > p$? — Les données découpées tiennent dans les caches : plus de swap.

Q7. $f = 5\,\%$ séquentiel : accélération maximale ? — $1/0{,}05 = 20$.

Q8. Peut-on avoir $P_{opt} < D_{max}$? — Oui : en déplaçant une tâche indépendante sur un créneau libre.

Q9. UMA vs NUMA ? — UMA : même temps d'accès partout. NUMA : dépend de la distance ; plusieurs SMP reliés ; extensible.

Q10. Cluster vs grille ? — Cluster : nœuds faiblement couplés sur LAN. Grille : clusters hétérogènes dispersés sur WAN/Internet, middleware.

Q11. Bus vs crossbar ? — Bus : un seul accès à la fois. Crossbar : accès simultanés vers des bancs différents, mais $p \times m$ switchs.

Q12. Quand y a-t-il conflit dans une boîte d'échange ? — Quand deux processeurs demandent des positions différentes (straight vs exchange).

Q13. Grille 2D 4×5 : n, m, D ? — $n = 20$; $m = 40 - 9 = 31$; $D = 7$.

Q14. Hypercube de dimension 4 : n, m, D, Δ ? — 16 ; 32 ; 4 ; 4. Distance entre 0110 et 1011 : 3.

Q15. Pourquoi l'étoile est-elle infaisable à grande échelle ? — Le centre a un degré $n - 1$.

Q16. Chaîne de $p = 8$ processeurs, o2a depuis $i = 3$ en link-bound ? — $\max(2, 5) = 5 \Rightarrow T = 5\left(\tau + \frac{N}{\beta}\right)$.

Q17. Anneau de 10 processeurs, o2o de 1 vers 4 sur un seul chemin ? — $\min(3, 7) = 3 \Rightarrow T = 3\left(\tau + \frac{N}{\beta}\right)$.

Q18. Grille 2D, o2o de $(1, 2)$ vers $(3, 5)$ sur deux chemins ? — distance $2 + 3 = 5 \Rightarrow T = 5(\tau + N/2\beta)$.

Q19. Processor-bound vs link-bound ? — Un seul envoi à la fois par processeur, contre des envois simultanés sur tous les liens.

Q20. Par quoi une tâche est-elle définie ? — $(E_i, S_i, F_i, \theta_i)$; indivisible, un seul processeur, non préemptive.

Q21. $T_1 : x = y + z$ et $T_2 : w = 2y$ sont-elles indépendantes ? — Oui : les trois intersections sont vides (lire la même variable ne crée pas de conflit).

Q22. Qu'est-ce qu'une dépendance redondante ? — Un arc déjà déduit par transitivité ; on l'élimine du graphe.

Q23. `private` vs `firstprivate` ? — `private` n'est pas initialisée ; `firstprivate` prend la valeur d'origine.

Q24. `master` vs `single` ? — `master` : thread 0, pas de barrière ; `single` : un thread quelconque, les autres attendent.

Q25. Quel `schedule` pour des itérations de coûts très inégaux ? — `dynamic` (ou `guided`).